

Reading the Moon

A Public Science Platform

An Intensive 3-Week AI Literacy Curriculum Through Lunar Science

The Moon is my beautiful impossibility.

Ping Wang

April 2026

[Tutorials](#) • [Datasets](#) • [Exhibits](#) • [Colab Notebooks](#)

Program Vision

Reading the Moon: A Public Science Platform and Curriculum is built on the premise that interested members of the public can participate in the frontier of planetary science and AI, not merely observe it. The platform connects two strands:

1. **Strand A: Using AI to investigate the Moon**
2. **Strand B: Building and understanding AI with lunar data**

Strand A guides learners through a staged progression from station 0 to 6. Learners work the stations: they identify major minerals in rotating thin-section imagery, generate labeled datasets, train and interpretable K-nearest-neighbor classifier, segment mineral grains using SLIC phase-segmentation annotator, build knowledge base using the lunar olivine oxide data extractor, and interrogate a lunar sample agent as a web-augmented literature search and synthesis. In doing so, learners experience what AI tools can and cannot do, including verifying extractions against Apollo-era PDFs and catching errors, and watching the agent’s tool trace. The verified outputs of station work, including phase annotations, lunar olivine oxide CSVs checked against printed page, corrected markdown conversions of legacy papers, flow into a shared, versioned lunar sample corpus. Thus, learners contribute to scientific work during learning: the digitization and verification of decades of lunar sample literature. The core stations operate on local hardware without continuous internet connectivity, reducing access barriers for schools, rural communities, and resource-constrained venues. Thus, the platform offers a template for public participation in education and research at the moment AI makes it possible.

Strand B equips learners to build, train, and evaluate AI models using lunar sample texts and customized datasets drawn from the platform. Through pre-built Colab notebooks, learners progress from a character-level SampleGPT to a BPE-tokenized SampleGPT, then explore post-training through supervised fine-tuning (SFT) and reinforcement learning (RL), comparing how each stage changes model behavior and performance. Learners also train and experiment with compact architectures such as NanoChat and MiniMind, using customized lunar datasets they curate or generate through the Reading the Moon platform and curriculum. The goal is to make the full model-development pipeline visible, from data preparation and tokenization to pretraining, fine-tuning, alignment, evaluation, and model comparison, while grounding every step in lunar science.

Spiraling, Hands-on AI Literacy through Lunar Science

The curriculum uses a spiraling and hands-on design that integrates AI literacy with lunar science and lunar sample datasets. Learners repeatedly revisit the same core AI concepts, including data, representation, prediction, training, evaluation, and alignment, while working with increasingly sophisticated models and increasingly rich scientific data at increasing levels of technical depth. At each stage, learners learn by doing: exploring lunar data, curating datasets, running models, changing hyperparameters, training language models, comparing outputs, identifying failures, and contributing improved data and models back to the platform. Equipped with pre-built Colab notebooks to train language models from scratch on lunar sample text, the curriculum aims to make an otherwise invisible AI system observable, manipulatable, and understandable through experience even though they might not understand every line of code. Well-designed Colab notebooks turn model training into a laboratory so learners do not need to understand all the code to understand what the AI system is doing, what choices shape it, and why its outputs have limitations while inspecting the lunar text used for training.

Curriculum Overview

- **Format:** 3 weeks •5 days per week •6 instructional hours per day (90 contact hours)
- **Audience:** high school students
- **Resources:** *Reading the Moon* Interactive Exhibit (<https://planetaryai.github.io/exhibit/index.html>)
- **Companion codebases:** Colab notebooks, Karpathy's minGPT, NanoGPT, NanoChat, character-level SampleGPT with curated lunar sample text, BPE-tokenized SampleGPT with curated lunar sample text
- **Companion tutorials:** Linear Algebra for High School Students, Chemistry Basics for Reading the Moon, Users' Guide on the Interactive Exhibit
- **Companion datasets:** curated lunar sample text, rotational series of Apollo lunar thin sections, lunar olivine in crossed polars

Learning Outcomes

The goal of the curriculum is to develop practical, critical and creative AI literacy through hands-on engagement with lunar sample data and customized datasets drawn from the platform. By the end of the curriculum, learners will be able to:

- Make and record systematic petrographic observations and clearly distinguish observation from interpretation.
- Write evidence-based petrographic reports that support interpretations of lunar history.
- Design a Lunar petrographic exhibit through artistic expression.
- Understand how AI systems are built, trained, evaluated, and improved.
- Explain and implement self-attention (queries, keys, values) and describe the modern LLM pipeline.
- Define a language model as a next-token predictor; implement one at the character-level and BPE-tokenized SampleGPT.
- Understand how training data, tokenization, model size, training objectives, and human preferences shape AI behavior.
- Build awareness that scientific AI depends on the quality of its data.
- Critically evaluate AI-generated outputs in a scientific setting: identify hallucination and articulate when AI assists and is not trustworthy.

Prerequisites, Materials, Compute

- Students: No prior coding required.
- Hardware: One laptop per student; reliable internet. Google Colab (free tier) covers 3 weeks.
- Corpus: a lunar sample text corpus, petrographic reports (<https://planetaryai.github.io/>)

PART 1: USING AI IN LUNAR SCIENCE

Framing Question: Where does an AI's knowledge come from?

Mornings (Blocks A and B) follow the exhibit stations in order. Afternoon (Blocks C and D) run the data & AI bootcamp, always using the mornings's scientific material as the dataset.

Day 1: Lunar Rocks are Simple

Exhibit station: 00 Intro

Objectives: Introduce Apollo lunar missions and lunar sample compendium; set up the coding environment.

Block A: Orientation & Apollo lunar mission. Program welcome, norms, team formation (pairs that persist all three weeks).

Block B: Station 00. Lecture: Why lunar rocks are simple? Introduce lunar sample compendium.

Block C: Bootcamp 1: environment and first programs. Colab setup; variables, types, lists, dictionaries, f-strings. All exercises use lunar content.

Block D: Studio: Text is data. Introduce prebuilt lunar-sample text based on the lunar sample compendium. Compute corpus statistics: character frequencies, corpus length, vocabulary size.

Day 2: Lunar Minerals

Exhibit station: 01 Mineral Lab

Objectives: Identify plagioclase, pyroxene, olivine, ilmenite, and glass by behavior under PPL and XPL; understand why lunar mineralogy is readable.

Block A: Hook: a rotating lunar thin-section GIF under PPL and XPL and ask *could a machine tell what this is? what would it need to know?* Lecture: mare basalt; polarizing microscope; diagnostic fingerprint of relief, cleavage, fracture, twinning, birefringence color, and extinction.

Block B: Station 01 deep exploration. Pairs work through the Mineral Lab, rotating the lunar thin sections on Virtual Microscope for each of the four minerals and completing a structured observation matrix: *how does each mineral behave, and which properties separate it from the other three?* Then, each pair presents one mineral to the class in three minutes. Debrief introduces AI vocabulary — the optical properties are *features*, the mineral name is a *label*.

Block C: Bootcamp 2: dictionary mapping minerals to their birefringence descriptions, etc; load rotational series of images into Pillow, CV2, matplotlib.

Block D: Studio: write a function that quizzes a partner on mineral properties. Journal prompt: *define feature in your own words and give a non-geology example.*

Day 3: Training Data

Exhibit station: 02 Dataset

Objectives: Build a labeled dataset; understand ground truth; learn loops and functions.

Block A: Labeling. Recap features /labels. Pairs label rotational series and record relief, birefringence, cleavage, fracture, twinning, and opacity, then assign a mineral by following the exhibit's instruction to be honest about what they actually see.

Block B: Labeling. Continue labeling for variety. Then a controlled experiment: each pair independently labels the same rotational series; the class tabulates results on the board. Where do annotators disagree, and on which optical properties? Introduce label noise. Export datasets as CSV and JSON.

Block C: Bootcamp 3: loops, functions, conditions. Write functions that operate on observation records.

Block D: Studio: Groups swap exported CSVs and audit each other: doubtful labels, missing variety. Each group writes a one-page data card: how the data was collected, by whom, known weaknesses, and recommended use. Discussion: what happens to a model trained on flawed labels?

Day 4: Train, Evaluate, Deploy

Exhibit station: 03 ML

Objectives: Train a k-NN classifier on student data; understand held-out evaluation; load and plot data in code.

Block A: Lecture on How k-NN works, and then train it. Train with $k = 1, 3, 5$ and predict how each will differ before testing.

Block B: Run the evaluation on the 40 held-out slides the model has never seen. Then the deploy the panel: classify live unknowns, correct mistakes, watch the learning curve and live-accuracy readout, re-run the held-out evaluation, and compare before/after. Class leaderboard, followed by the more important discussion: trace each group's accuracy back to their dataset size, variety, and label quality.

Block C: Bootcamp 4: NumPy, pandas, matplotlib. Load your own exported CSV into pandas; compute label counts; scatter-plot two features colored by mineral; see your training set as geometry. Concept bridge: distance in feature space is what k-NN sees.

Block D: Studio: build k-NN from scratch.

Day 5: Phase Segmentation

Exhibit station: 04 Annotator

Objectives: Contrast feature-based and language-based reasoning; experience pixel-level image annotation.

Block A: Discussion: the k-NN reasons from numbers; the LLM reasons from words you wrote — but where did the LLM’s knowledge of petrography come from in the first place? Collect hypotheses on the board, seal them in an envelope, and promise the answer in Week 2–3.

Block B: Students upload a rotational series and ask the AI Claude to identify it and explain its reasoning; then compare the verdict and explanation against their own k-NN’s output on the same unknown. Structured comparison sheet: Which agent agreed with ground truth? Whose explanation was more useful? Which claims could you verify, and which couldn’t you? Every unverifiable claim goes on a public "Claims Wall" that stays up all three weeks. Label mineral grains using superpixels. Mini-lecture: image segmentation and the economics of annotation — vision datasets are built by slow, careful human labor at enormous scale; annotation quality bounds model quality.

Block C: Bootcamp 5: Label mineral grains using superpixels. Mini-lecture: image segmentation and the economics of annotation — vision datasets are built by slow, careful human labor at enormous scale; annotation quality bounds model quality.

Block D: Studio: compute a simple pixel-level statistic (mean color per superpixel) in the notebook. Journal: *name one thing the LLM said that you could not verify — how would you check it?*

Day 6: Extracting Structured Data in Apollo-era PDFs

Exhibit station: 05 Extractor

Objectives: Extract structured olivine oxide data from Apollo-era PDFs. understand persistent memory.

Block A: Lecture on lunar olivine oxide compositions.

Block B: Use the Extractor to pull lunar olivine oxide data out of scanned Apollo-era PDFs. Discussion: Decades of scientific knowledge is locked in documents. Students spot-check extracted values against the source PDFs and log any discrepancies (a concrete taste of validation).

Block C: Bootcamp 6: Rules-based extraction of lunar olivine oxide data from markdown files.

Block D: Studio: Groups build a poster-sized concept map of the full pipeline, annotating every stage with the error modes they personally witnessed. Gallery walk with structured peer feedback.

Day 7: Lunar Sample Agent

Exhibit station: 06 Agent

Objectives: Understand agents as model+tools+loop.

Block A: Explore the agent working in real time: it plans, calls tools (search, extraction, classifica-

tion), observes results, and acts again.

Block B: Students assign the Agent tasks and trace each step in a structured log. Vocabulary: `**agent = model + tools + a loop.**` Discussion: which failures in the loop would be caught, and which would compound?

Block C: Bootcamp 7: Review the code for the lunar sample agent.

Block D: Studio: Corpus preview: the class inspects the lunar-text corpus, including Apollo sample descriptions and petrographic text, and predicts what a language model could learn from it. Closing teaser, live-demoed: a model that has read this text will write new lunar sample descriptions, one character at a time.

PART 2: BUILDING AND UNDERSTANDING AI WITH LUNAR DATA

Framing question: How does an LLM actually work?

An LLM is a next-token prediction machine; fluency, knowledge, and hallucination all emerge from that single objective.

Day 8: Text is Data

Objectives: Clean a corpus; map characters to integers.

Block A: Download lunar sample reports in lunar sampel compendium; and examine headers, page numbers, tables and figures. Students write cleaning code: normalize whitespace, strip junk lines, lowercase or preserve case (debate the tradeoff).

Block B: Build the character vocabulary; create 'stoi'/'itos' mappings; encode the corpus into a long integer sequence. Discussion: what did we just decide? (That the atomic unit of language is a character is a choice.) Compute corpus statistics: character frequencies, corpus length, vocabulary size.

Block C: Bootcamp 8: Review the code. Key idea of the day: A language model's entire job is answering *given what came before, what comes next?*

Block D: Studio: Corpus preview: the class inspects the lunar-text corpus, including Apollo sample descriptions and petrographic text, and predicts what a language model could learn from it. Closing teaser, live-demoed: a model that has read this text will write new lunar sample descriptions, one character at a time.

Day 9: Training a Character-based SampleGPT

Objectives: Train character-based SampleGPT with the prebuild Colab notebook and lunar sample text. Understand pretraining. Interact with a model the class watched come into being.

Block A: Tour the char-based SampleGPT. Walk the complete SampleGPT pipeline.

Block B: Training run. Two tracks depending on compute:

- Colab-T4
- Using CPU only

Block C: Bootcamp 9: Chat with what you built. Key idea: a chatbot is a next-token predictor.

Block D: Studio: Free exploration: sampling temperature effects; compare hallucination and documentation. Write the model card. Journal prompt: *what surprised you most about your language model?*

Day 10: Training a BPE-tokenized SampleGPT

Objectives: Train BPE-tokenized SampleGPT with the prebuild Colab notebook and lunar sample text. Interact with a model the class watched come into being.

Block A: Tour the BPE-tokenized SampleGPT. Walk the complete pipeline.

Block B: Training run. Two tracks depending on compute:

- Colab-T4
- Using CPU only

Block C: Bootcamp 10: Tokenization at the character level; create stoi/itos mappings; encode the corpus into a long integer sequence. Compute character frequencies, corpus length, vocabulary size. Tokenization at subword level. Tiktoken.

Block D: Studio: Free exploration: compare two language models. Use customized dataset to train char-based GPT and BPE-tokenized GPT.

Day 11: Attention is All You Need (1)

Objectives: Build an intuitive and mathematical understanding of how a sequence goes from tokens to contextualized representations through self-attention. Introduce Q,K,V. Understand why attention mechanism creates three representations from every token.

Block A: Lecture and discussion on: *what is the difference between a token ID and token embedding? What does the Transformer need positional information?*

Block B: Lecture and discussion on: *what does a query represent? what does a key represent? what does a value represent?*

Block C: Bootcamp 11: use a tiny sequence of four tokens to understand QKV.

Block D: Studio: Free exploration and important distinction: *keys are used to determine attention*

weights. values are used to construct the output.

Day 12: Attention is All You Need (2)

Objectives: Build an intuitive and mathematical understanding of how a sequence goes from tokens to contextualized representations through self-attention. Compute attention scores. Understand how a query decides which tokens are relevant. Turn raw similarity scores into attention weights. As vector dimension increases, dot products can become large. Scaling keeps the values numerically manageable before softmax. Attention weights are applied to values. Connect attention to the a contextualized token representation.

Block A: Lecture on how to compute attention scores.

Block B: Lecture on how to turn raw similarity scores into attention weights. Scaling and softmax.

Block C: Bootcamp 12: use a tiny sequence of four tokens to understand attention mechanism in a transformer.

Block D: Studio: Free exploration: one attention head might learn relationships related to grammar, nearby words, subject-verb relationships, pronoun references, semantic similarity, long-range dependencies.

Day 13: Attention is All You Need (3)

Objectives: Build an intuitive and mathematical understanding of how a sequence goes from tokens to contextualized representations through self-attention. Multi-head attention.

Block A: Move from a single attention mechanism to the architecture actually used by Transformers. Transformers create multiple attention heads. The outputs are concatenated.

Block B: Lecture and discussion on: *how does attention make a token representation contextual? why are multiple attention heads useful?*

Block C: Bootcamp 13: implement the self-attention mechanism from scratch with prebuilt Colab notebook.

Block D: Studio: Free exploration and important distinction: *why are attention weights multiplied by V , rather than K ? how does attention make a token representation contextual?* Experiment with attention weights and value vectors to discover how self-attention transforms a token's initial embedding into a context-dependent representation. Ask question: *what happens if one attention weight becomes dominant? what happens if attention is uniform? which contextual representation changes when you replace one word in the sentence?*

Day 14: NanoChat

Objectives: Walk the complete modern LLM pipeline, understand pretraining vs supervised fine-tuning vs reinforcement learning.

Block A: NanoChat pipeline: tour the NanoChat codebase: tokenizer training, BPE, transformer, loss and training loop, evaluation. Large-scale pretraining data.

Block B: Lecture and discussion on supervised fine tuning on conversation data. Key idea: a chatbot is a next-token predictor finetuned on conversations formatted as text.

Block C: Bootcamp 14: Training run. Watch loss curves and logging milestones. Pretrain a scaled-down NanoChat configuration, resume from pre-trained base checkpoint, run the finetuning stage which is small enough to complete. See the base model's raw completions before finetuning vs its chat behavior after.

Block D: Studio: Chat with what you built. Serve the model pretrained. Structured comparison protocol: pose identical prompts and score fluency, knowledge, instruction-following, and uncertainty. Key idea: capability tracks scale. Reading NanoChat's finetuning data format and RL data format. Journal prompt: *what surprised you most about how little separates your model from a chatbot?*

Day 15: Finetuning a Small Language Model with Customized Lunar Dataset(s)

Objectives: Finetune a small language model on a custom lunar-domain dataset so that it becomes more capable at answering questions, following instructions, or generating text about the Moon, lunar science, lunar missions, and related topics.

Block A: Lecture and discussion on: *what changes when a pretrained language model is finetuned?*: pretraining learns broad language patterns from massive datasets. Finetuning adapts to a smaller specialized dataset.

Block B: Lecture and discussion on RAG with lunar olivine PDFs used in Station Extractor.

Block C: Bootcamp 15: Build the custom lunar dataset using lunar mission descriptions. Design the training format and understand how the same information can be represented differently depending on the training objective. Key idea: the dataset teaches not only what the model knows but also how the model should respond.

Block D: Studio: Use prebuilt Colab notebook to finetune a small language model and Chat with what you built. Journal prompt: *what surprised you most about how little separates your model from a chatbot?*